

SG-Lean 思路汇报

面向 Lean 4 的形式化数学证明器

文档说明：本文件说明项目的目标、总体思路、执行流程与工具模块，是当前思路的权威口径。
仓库中若干历史文档（README.md、docs/framework.md 等）描述的是代码实现的演变过程，
若与本文件冲突，以本文件为准。

目录

1	目标与指标	2
1.1	项目目标	2
1.2	指标与优先级	2
1.3	范围与边界	2
2	思路框架与执行流程	2
2.1	总体框架	2
2.2	与 SGS 的关系	3
2.3	执行流程一：在线求解一道题	4
2.4	执行流程二：库的建立与管理	5
2.4.1	建立流程	5
2.4.2	库的管理规则	5
3	工具模块	7
3.1	模块总览	7
3.2	Lean 侧工具	9
3.3	Python 侧工具	9
3.4	复用与上游	9
3.5	待建模块	9
3.6	数据角色与评测	10
A	术语对照	10

1 目标与指标

1.1 项目目标

本项目的目标是构建一个 **Lean 4 形式化数学证明器**：输入一条数学命题的 Lean 形式化陈述，输出一段能够通过 Lean 内核检验的证明脚本。

输入	一条 Lean 命题。可以是 <code>theorem</code> 或 <code>example</code> 语句（其证明体为 <code>sorry</code> ），从语句头部抽取绑定变量转换为全称量化的闭式命题；也可以是等价的闭式命题（只引用全局常量）。
输出	一段 tactic 证明脚本，外加验证记录（内核实测通过、耗时、来源后端等）。
保证	输出的每一篇证明都经过 Lean 内核终检。未通过验证的候选证明 不会 出现在输出中。

1.2 指标与优先级

项目的全部工程取舍，按下列顺序排列优先级：

优先级	指标	定义与说明
1	正确率（主指标）	在固定评测集上报 pass@1 与 pass@k ， 目标级统计 ：一道题只要有至少一篇候选通过内核检验，即算该题被解出。
2	模型调用成本	token 消耗量与调用次数。任何新增杠杆都必须报出它的边际成本。
3	时间成本	墙钟时间，含 Lean 侧验证的等待（Lean 批处理的固定开销较大，需单独核算）。

这一顺序意味着：当两条技术路线在正确率上无法区分时，选成本低的那条；当一条路线要花更多调用换取少量正确率提升时，需要有实测数据支撑才会被采纳。

1.3 范围与边界

- **不做模型训练**：不训练、不微调基座模型。证明能力的上限由求解模型决定，本项目贡献的是模型之上的脚手架，以及用 Lean 保证的输出正确性。
- **不做自动形式化**：不处理自然语言题干到 Lean 命题的转换，输入即为 Lean 命题。
- **不承诺通用性**：只承诺在选定评测基准与 Mathlib 覆盖范围之内尽可能高的正确率；超出范围的命题能力仅由基座模型决定。
- **搜索暂为储备**：证明搜索（MCTS 一类）作为可选后端保留，当前不作为主线。

2 思路框架与执行流程

2.1 总体框架

整体上，本项目由**两条时间尺度**构成，二者通过**引理库**这一个接口相连：

- **在线求解**（秒级至分钟级）：接收一道题，输出一篇通过验证的证明。
- **离线资产生产**（小时级）：在一批课程集上运行闭环，把重复出现的能力缺口转化为已通过验证的引理，沉淀为库。

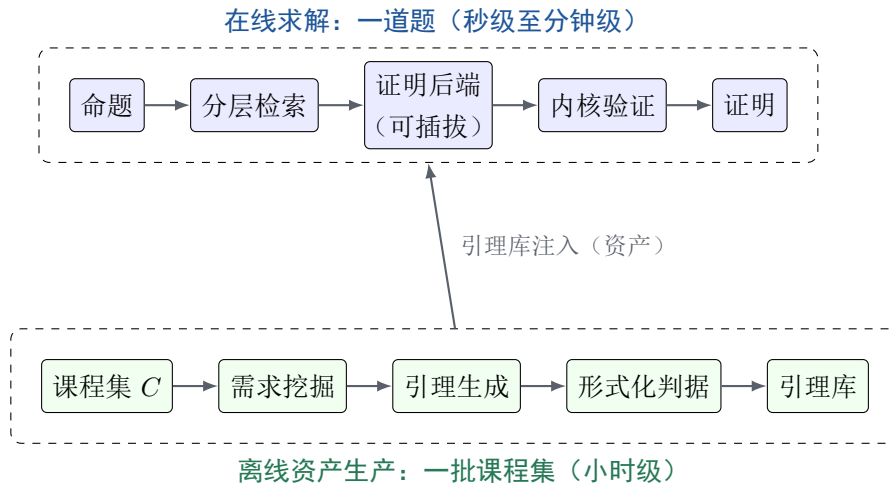


图 1: 总体框架：在线求解与离线资产生产，通过引理库相连。

三条不变量（框架的骨架，不做妥协）：

1. **能力来自基座模型。**我们不训练大模型，因此不承诺通用性，只承诺在基准与 Mathlib 覆盖范围内尽可能高的正确率。
2. **增益来自可寻址的资产，而不是权重。**资产具备两个性质：**可加**（新增一个领域的引理不会破坏已有领域的引理）与**可寻址**（解决一道题时只需要其中相关的少数几条）。这是应对未知题型的结构性优势。
3. **正确性由 Lean 内核保证，不由模型保证。**任何来源的候选证明，都必须经过同一道内核闸门。

2.2 与 SGS 的关系

本项目的思路源于 SGS (*Scaling Self-Play with Self-Guidance*) 的三角色自博弈框架：求解者 (Solver)、出题者 (Conjecturer) 与评审者 (Guide)。我们保留其结构、循环形状与防退化思想，替换其中两样东西：

两点必须如实说明：

1. **SGS 的循环依靠梯度闭合。**本项目不训练权重，因此本项目的形态对应于 SGS 消融实验中的“冻结出题者”一档。SGS 论文的原话是其表现优于纯强化学习基线，但劣于完整方法。我们以“资产累积”承担原本由权重更新承担的角色，这一替代是否充分，正是本项目要回答的问题。
2. **SGS 的评测口径是在训练题上统计累计解出率；**本项目采用留出集协议（课程集建库、开发集调参、测试集冻结后只跑一次），在评测严谨性上比 SGS 更严格。

SGS（训练期）	本项目（推理期）
Guide 由大模型对合成题打分	替换为 形式化判据 $G = \text{硬门} \times \text{软分}$ ，全部由 Lean 计算，确定性、不消耗 token
用强化学习更新求解者与出题者的权重	替换为 引理库的更新 ，并以提示词注入的方式在下一轮被使用
出题者条件化在未解目标上	保留，并扩展输入：未解目标 + 需求签名 + 库中已入库引理范例
奖励为 $R_{\text{solve}} \cdot R_{\text{guide}}$	保留乘法结构：可证性以硬门形式保留，质量以软分（压缩收益与覆盖收益）体现

2.3 执行流程一：在线求解一道题

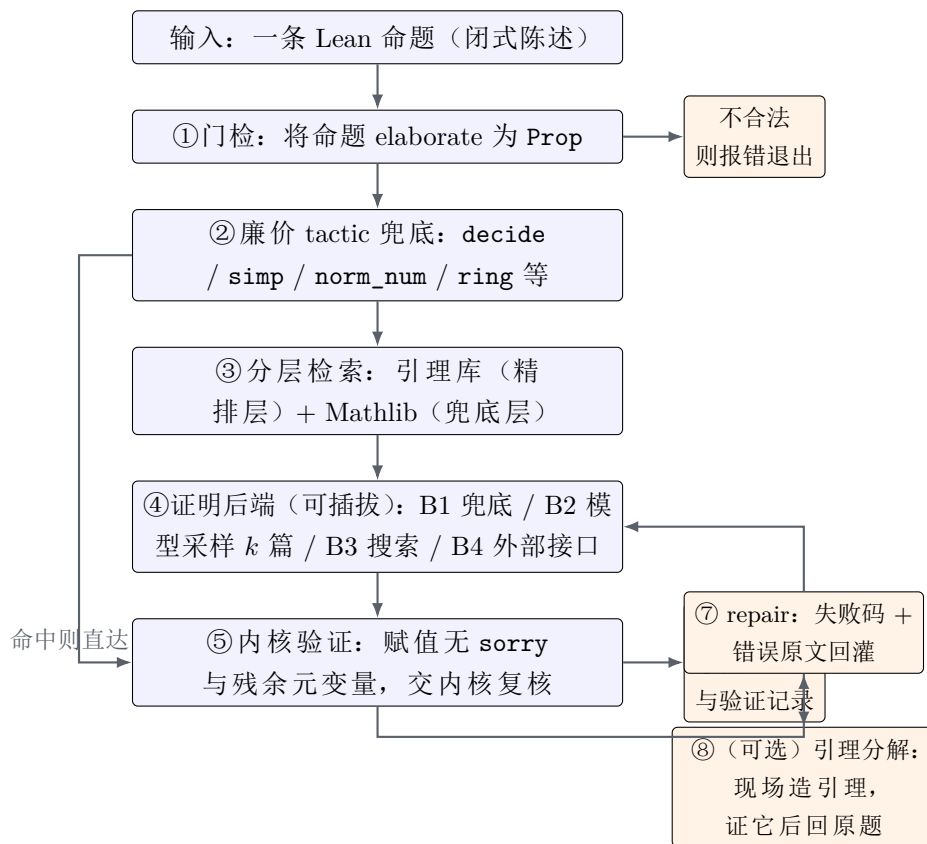


图 2：在线求解流程：从命题到通过内核检验的证明。

四个设计要点：

1. **后端可插拔**。求解方法不固定，任何候选证明来源都可以接入，因为所有后端共享同一道内核闸门，正确性不依赖后端。
2. **级联按成本排序**。先执行零成本的兜底，再执行有调用成本的模型采样，最后才考虑高成本的搜索。级联顺序由各后端的“边际正确率 / 边际成本”决定。
3. **分层检索而非二选一**。自建库是精排层，Mathlib 是兜底层。库未覆盖的题型会自动回落到 Mathlib，系统不会因库的不足而失效。

步骤	模块	做什么与产出什么
①	门检 (Lean)	判断命题是否为合法命题。不合法直接报错退出，这是题目的问题，而不是“证不出来”
②	廉价 tactic 兜底 (Lean)	在预算内依次尝试若干自动化 tactic。命中时该 tactic 本身即为证明（很多题根本不需要调用模型）
③	分层检索 (新增)	第一层在引理库中按相关度取前若干条；库里没有的，回落到 Mathlib 前提检索
④	证明后端 (可插拔)	统一接口：给定命题，返回若干候选证明脚本。后端可增删替换，例如本地 tactic、语言模型采样、证明搜索、外部自动证明器
⑤	内核验证 (Lean)	造孤立义务目标，执行候选证明脚本，最后检查赋值中无 <code>sorry</code> 与残余元变量，并交内核复核
⑥	输出	仅输出通过⑤的证明脚本，并附验证记录
⑦	repair (新增)	全部候选失败时，把结构化失败码与 Lean 错误原文回灌给证明后端重试，受轮数与 token 预算限制
⑧	引理分解 (可选)	针对本题现场生成辅助引理并证明，再回到原题。这些临时引理 不进入全局库

4. **失败信息是资产。**内核返回的是结构化失败码（未知标识符、类型错误、目标未闭合、含 `sorry`、超时等）与错误原文，比单纯的“失败”信息量大得多，可直接用于定向重试。

2.4 执行流程二：库的建立与管理

2.4.1 建立流程

库不在解题时现造，而是由课程集离线跑闭环产出。

闭环中，三个角色分工明确：

- **求解者**在课程集上生成证明并留下轨迹；
- **出题者**针对需求缺口生成引理候选；
- **评审者**的职责由形式化判据承担，即硬门与软分，决定哪些候选入库。

注意：出题者与评审者**只存在于离线环节**；在线解一道题时它们不出场。

2.4.2 库的管理规则

（一）**两种来源。**库同时接收两类引理：精选层，即从 Mathlib 检索筛选出对特定场景特别有用的引理，成本低、优先做；自造层，即过程中新证明出来的命题，成本高、后做，但它是产生新数学内容的唯一入口。

（二）**四条准入规则：**

1. 自造引理必须通过同一套硬门。“已经被证明”只满足可证这一条，还要通过非平凡与新颖的检验。

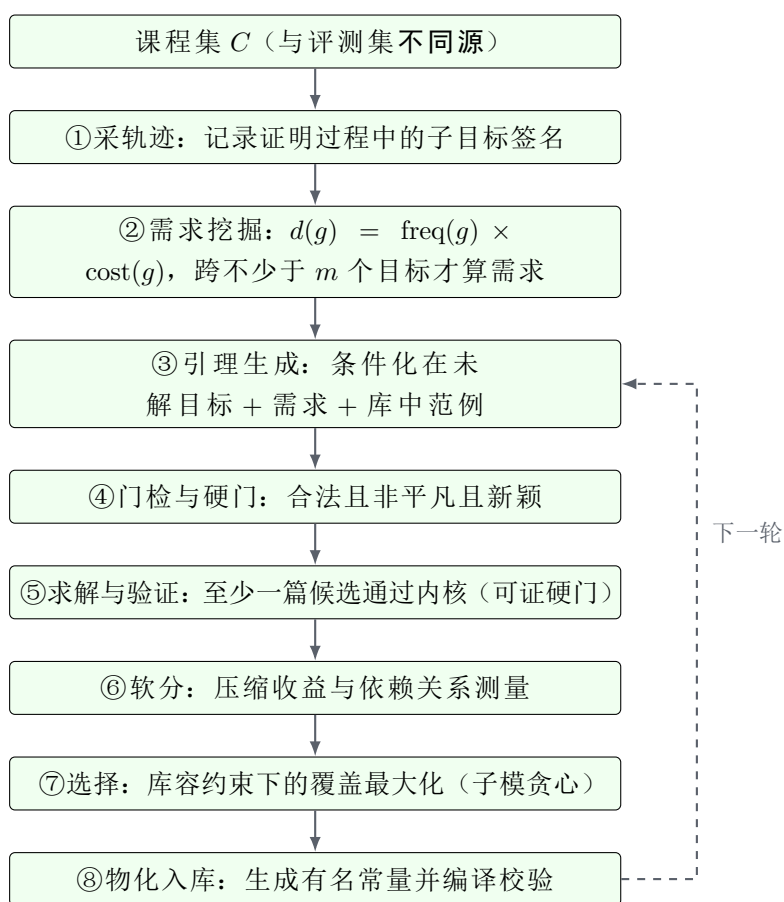


图 3: 库的建立流程：课程集经过八步转化为可编译的引理库。

2. 必须记录来源信息（在哪道题、哪一轮产生），否则无法在评测时判断是否发生泄漏。
3. “常用”必须有实测依据，即实测被依赖率与覆盖增益，不能凭印象判断。
4. 入库前先与 Mathlib 去重。若自造引理与 Mathlib 中已有命题等价，不必重复入库，而应把这条对应关系存为**推荐映射**，告诉求解器“这类题优先考虑那条引理”。这本身比存一条重复引理更有价值。

（三）**权重与排序**。“常用”不能直接等同于被引用次数：若权重只按引用次数累计，而引用机会又取决于排序，就会形成正反馈，使少数几条垄断呈现机会、新引理永远上不来。因此采用

$$\text{权重} = \frac{\text{被引用次数}}{\text{被展示次数}} + \text{探索额度}.$$

即类似点击率而非单纯点击量，并给新入库引理一个初始展示额度，保证它有机会被验证是否有用。

（四）**淘汰与版本**。库不能只增不减。需要定期在工作负载上重跑评测，把权重低于阈值的引理降级到冷存（保留记录，不直接删除）。每次入库同时保存库的版本快照，否则实验不可复现。

（五）**六条污染路径与对策：**

污染路径	对策
重复入库：同一道题反复运行，等价引理反复进库	新颖性硬门对库内去重
过拟合少数题：库中引理全部围绕两三道题	准入门槛加跨目标计数，一条引理至少要能帮到两个不同目标
语料来源单一：课程集风格高度同质	课程集按子领域与题型分层抽样
过拟合工作负载：反复用同一批题挑选引理	工作负载只用于挑选，并留出一部分不参与挑选、只用于验证
评测污染：用测试集建库	数据角色硬性隔离，并记录来源信息
自我强化：呈现机会被少数引理垄断	权重按展示次数归一，并保留探索额度

（六）**冻结原则**。评测时库冻结，解题过程中不再更新；在线环节现场生成的临时引理不进入全局库。否则所测得的增益属于自我循环，结论不可用。

3 工具模块

3.1 模块总览

下表中，Lean 侧模块的路径以 `sgslean/` 为根，Python 侧模块的路径以 `sgsr/` 为根，实验脚本位于仓库的 `scripts/` 目录下。

模块	位置	职责与状态
门检	SgsLean/Gate.lean	判定命题是否合法；已实现
内核验证	SgsLean/Verify.lean	整篇证明的重放与内核终检；已实现
非平凡性	SgsLean/Trivial.lean	判定命题能否被自动化 tactic 秒杀；已实现（可复用为在线兜底臂）
新颖性	SgsLean/Novelty.lean	与父目标及库内命题的等价性查重；已实现
软分测量	SgsLean/Measure/	压缩收益与依赖关系抽取；已实现，接线中
物化与库	SgsLean/Materialize.lean pipeline/library.py	生成有名常量、编译校验、库的读写与统计；已实现
轨迹与需求	SgsLean/Trace.lean pipeline/demand.py	子目标签名与需求挖掘；已实现
覆盖与选择	pipeline/coverage.py	覆盖度计算与子模贪心选择；已实现
闭环编排	pipeline/runner.py	十步一轮的建库编排；已实现（离线冒烟通过）
模型服务	models/	真代理、假服务、提示词与解析、OpenAI 兼容后端；已实现
Lean 服务	SgsLean/Server.lean verification/client.py	标准输入输出 JSON 协议，支持批处理与预算配置；已实现
证明器入口	pipeline/prover.py scripts/prove.py	待建：单题进、证明出的统一入口
检索层	[位置待定]	待建：Mathlib 前提检索与库的按需取用
repair 回路	models/prompts.py	待建：失败信息的回灌与重试

3.2 Lean 侧工具

Lean 侧是本项目的正确性基础,位于 `sgs-reap/sgslean/`,以 lake 包形式组织,依赖 Mathlib。它以标准输入输出 JSON 服务的形式对外提供能力:

- 命题合法性检查与命题到类型表达式的 `elaborate`;
- 整篇证明的验证 (含内核终检);
- 非平凡性与新颖性判定;
- 压缩收益与依赖关系测量;
- 已验证引理的物化 (生成有名常量) 与编译校验;
- 子目标轨迹记录。

验证链路的关键在于: 候选证明先被包装成 `tactic` 执行,随后检查赋值中是否存在 `sorry` 与残余元变量,最后交给 Lean 内核复核。需要特别指出,在本版 Lean 中 `sorry` 的语法节点与上游守卫所匹配的名称不一致,仅靠语法检查会漏判,真正拦住它的是内核终检这一步。

3.3 Python 侧工具

Python 侧位于 `sgs-reap/sgsr/`, 按职责分为四个子包:

- `models/`: 模型服务。包含真实代理 (提供生成、评审、求解等端点)、确定性假服务 (离线测试用)、提示词与解析器、OpenAI 兼容后端、配置读取。
- `pipeline/`: 流水线。包含需求挖掘、引理生成组织、库的读写、覆盖度与选择、以及闭环编排。
- `verification/`: Lean 服务的常驻客户端,负责批处理与预算传递。
- `data/` 与 `utils/`: 数据模式定义,以及服务启动与日志辅助。

3.4 复用与上游

项目在 `sgs-reap/reap-fork/` 中维护了一个上游证明器项目的本地分支。其中**验证内核是本项目的地基**: Lean 侧的门检、验证、轨迹与非平凡性判定都直接建立在它的证明重放与内核检查原语之上。此外,上游完整的证明搜索实现 (含搜索树与超参数配置) 已经就绪,但默认关闭,作为可选的证明后端保留。

3.5 待建模块

1. **证明器入口**: 接收一条命题或一个 `.lean` 文件,串起“门检、兜底、检索、后端采样、验证、重试”,并写回通过的证明。这是本项目作为产品的本体,同时也是最终评测的入口。
2. **检索层**: 现状是将库整体注入提示词,这在库规模增大后不可行 (上下文装不下,且无关引理会变成噪声)。需要改为按相关度取前若干条,并接入 Mathlib 前提检索作为兜底。

3. **repair 回路**: 把结构化失败码与错误原文写回提示词, 驱动定向重试。失败信息在上游已经产出, 目前尚未被利用。
4. **兜底臂**: 把非平凡性判定所用的一组 tactic 从“返回是否秒杀”改造为“返回证明脚本”, 使其在线流程中直接产成证明。

3.6 数据角色与评测

数据按角色严格隔离, 这条规则不可违反:

数据集	角色	约束
课程集 C	建库: 需求挖掘、引理生成、入库	必须与评测集不同源; 这是唯一可以用于建库的数据
开发集 D	调参: 选择工作负载、确定采样数、观察方向	可用于反复实验, 但报告中必须说明所有超参数均在开发集上确定
测试集 T	最终评测	框架与超参数冻结后只运行一次, 结果无论好坏均如实报告

评测按**领域分桶**报告正确率, 以便观察在未见领域上的表现是否存在塌陷。需要提前注意一个来源风险: 常用竞赛数学基准中的部分题目来自公开数学题库, 因此该题库不能用作课程集; 另一个候选语料来源与竞赛题库存在聚合关系, 使用前必须先与评测集查重。若不做这一步, 所测得的增益将因数据泄漏而不可用。

关于语料的规模与难度, 有一条来自实测的经验: 库能起作用的地方是求解器 **差一口气**的地方。完全解不出的题目上无法测出增益, 过于简单的题目又不需要引理。真正有信号的区间是解出率介于零与一之间的“近失手”题目, 而这类题在常用基准中占比约为一成。因此课程集的筛选应当以解出率分布为依据, 把难度瞄准在求解器能力边界附近, 而非一味追求难度。

A 术语对照

为避免同名异义, 下表给出本项目与 SGS 之间关键术语的对应关系。

术语	在 SGS 中	在本项目中
求解者	语言模型生成整篇证明	相同，仍然是语言模型。Lean 不是求解者，而是裁判
评审者	语言模型对合成题打分	被替换：由 Lean 计算的形式化判据，即硬门乘以软分，另含两臂覆盖测量
出题者	条件化在未解目标上生成训练题	生成引理候选，输入扩展为需求与库中范例
验证	由 Lean 编译器判定	相同，但更严格：含内核终检与元变量检查
循环粒度	一批训练题，通过权重更新变强	分为两个尺度：离线建库循环（一批课程集，更新库）与在线求解（单题一趟加修复）
反馈流向	评审分数进入训练，更新出题者权重	分数流向选择层，决定哪些引理入库；回灌给出题者的只有库中范例与需求